

String Input/Output Functions



The file then each time we will need to call character input function, instead C provides some string input/output functions with the help of which we can read/write a set of characters at one time.



Key Functions



fgets()

Reading a string from input

fputs()

Writing a string to output

1/4

fgets() and fputs() Functions

These functions are used to read and write strings.



Syntax

```
int fputs(char *str, FILE *stream);  
char *fgets(char *str, int num, FILE *stream);
```

The integer parameter in `fgets()` is used to indicate that at most `num-1` characters are to be read, terminating at end-of-file or end-of-line. The end-of-line character will be placed in the string `str` before the string terminator, if it is read. If end-of-file is encountered as the first character, EOF is returned, otherwise `str` is returned.

The `fputs()` function returns a non-negative number or EOF if unsuccessful.



Example

```
/*Program to read a file and count number of lines in file */  
#include <stdio.h>  
#include <conio.h>  
#include <process.h>  
#include <string.h>
```

```
void main()  
{  
    FILE *fp;  
    int cnt=0;
```

```
char str[80];
/* open a file in read mode */
if ((fp=fopen("lines.dat","r"))== NULL)
{
    printf("File does not exist\n");
    exit(0);
}
/* read file till end of file is encountered */
while(!feof(fp))
{
    fgets(str,80,fp); /*reads at most 80 characters in str */
    cnt++; /* increment counter after reading a line */
}
/* print number of lines */
printf("The number of lines in the file is :%d\n",cnt);
fclose(fp);
}
```

Block Input/Output Functions

Block Input/Output functions read/write a block from/to a file. A block can be a record, a set of records, or an array. These functions are also defined in the standard library.



Key Functions



fread()

Reading a block of data



fwrite()

Writing a block of data

In case of `fread()`, `buf` is a pointer to a memory area that receives data from the file and in `fwrite()`, it is a pointer to information to be written to the file. These functions are quite helpful in case of binary files.

Generally these functions are used to read or write an array of records from or to a file.





Example

```
/* Program to illustrate fread() and fwrite() functions */
#include <stdio.h>
#include <conio.h>
#include <process.h>
#include <string.h>

void main()
{
    struct stud
    {
        char name[30];
        int age;
        int roll_no;
    }
    s[30],st;
    int i;
    FILE * fp;
    /* opening a file in write mode */
    if((fp=fopen("sud.dat","w"))== NULL)
    {
        printf("Error while creating a file\n");
        exit(0);
    }
    /* reading an array of students */
    for(i=0;i<30;i++)
        scanf("%s %d %d",s[i].name,s[i].age,s[i].roll_no);
    /* writing to a file */
    fwrite(s,sizeof(struct stud),30,fp);
    fclose(fp);
    /* opening a file in read mode */
    fp=fopen("stud.dat","r");
    /* reading from a file and writing on screen */
    while(!(feof(fp)))
    {
        fread(&st,sizeof(struct stud),1,fp);
        fprintf("%s %d %d",st.name,st.age,st.roll_no);
    }
}
```

```
fclose(fp);  
}
```

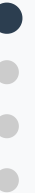
3/4

Sequential Vs Random Access Files

C supports two types of files – text and binary files, also two types of file systems – buffered and unbuffered file system.

It can also differentiate in terms of type of file access as Sequential access files and random access files. Sequential access files allow reading data from a file in sequential manner which means that data can only be read in sequence.

Random access files allow reading data from any location in a file. To achieve this purpose, C defines a set of functions to manipulate the position of the file pointer.





File Types



Text Files

Character data with translations



Binary Files

Data in same format as memory

The difference between these two files is in terms of storage. In text files, everything is stored in terms of text i.e. even if we store an integer 54; it will be stored as a 3-byte string - "54\0". In a text file certain character translations may occur.

A binary file contains data that was written in the same format used to store internally in main memory. The fact that a numeric value is in a standard length makes binary files easier to

handle. No special string to numeric conversions is necessary.

